

# 전자전기컴퓨터설계실험Ⅱ

## 실험 후 보고서

### LAB 1 – FPGA 조합논리회로

학과: 전자전기컴퓨터공학부

학번/이름: 2025440032 / 김윤지

작성일: 2026.09.18

대상: LAB1 조합논리회로

검증 환경: Vivado / FPGA board

프로젝트: ece2\_2026 / LAB1

GitHub 주소: [https://github.com/yunuos2525-cloud/ece2\\_2026](https://github.com/yunuos2525-cloud/ece2_2026)

제출 tag: LAB1\_POST\_SUBMIT

최종 code commit: 246d6b6

## 1. 실험 목적 및 검증 방법

조합논리회로는 현재 입력 조합에 따라 출력이 결정되며, 진리표나 논리식을 바탕으로 RTL 로 기술할 수 있다 [1]. LAB1 에서는 AND·OR·XOR 게이트, 가산기, 감산기, 비교기, MUX/DEMUX, 인코더/디코더, 7-segment decoder 등의 조합논리회로를 설계하고 검증하였다.

각 회로의 논리식과 변환 규칙으로 예상값을 계산하고, Vivado 시뮬레이션 파형의 안정 구간에서 입력·출력 관계를 확인하였다. 확보된 PASS 및 구현 기록은 시뮬레이션과 FPGA 구현 과정을 확인하는 보조 자료로 사용하였으며, 조교가 선정한 실험 06 MUX 와 실험 10 7-segment decoder 는 실제 보드 동작까지 비교하였다.

## 2. 실험별 설계 및 검증 결과

### 01. 논리 게이트

**설계 및 핵심 구조.** a 와 b 를 공통 입력으로 받아 AND, OR, XOR 결과를 각각 x, y, z 에 출력하도록 설계하였다. RTL 은 assign x=a&b, assign y=a|b, assign z=a^b 의 독립적인 연속 할당 세 개로 구성된다. 같은 입력이 세 연산에 병렬로 전달되어 각 게이트의 출력 차이를 비교할 수 있다.

**시험 조건.** 입력 (a,b)를 2'b00,2'b01,2'b10,2'b11 순서로 인가하였다. 특히 20~40 ns 에서 a=1'b1 을 유지하고 b 를 1'b0 에서 1'b1 로 바꾼 구간을 비교하였다.

**예상 결과.** x=a&b, y=a|b, z=a^b 이므로 입력 순서에 따른 {x,y,z}는 3'b000,3'b011,3'b011,3'b110 이다. 대표 구간에서는 3'b011→3'b110 을 예상하였다.

**관찰 결과.** 20~30 ns 에 x=1'b0,y=1'b1,z=1'b1, 30~40 ns 에 x=1'b1,y=1'b1,z=1'b0 이 나타났다. 앞선 입력 2'b00 과 2'b01 의 출력도 각각 3'b000 과 3'b011 이었다.

**해석.** 그림 1 에서 AND 는 두 입력 모두 1 일 때, OR 는 하나 이상의 입력이 1 일 때, XOR 는 두 입력이 다를 때 활성화되었다. 네 입력 조합에서 각 게이트의 진리표와 일치하였다.

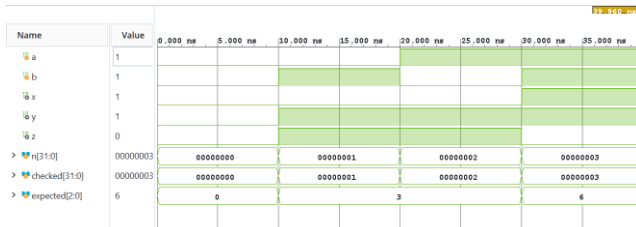


그림 1. 01. 논리 게이트의 입력·출력 및 예상값 비교 파형.

### 02. 전가산기

**설계 및 핵심 구조.** 두 개의 half\_adder 인스턴스를 연결하여 전가산기를 구성하였다. 첫 번째 반가산기는 a 와 b 에서 partial\_sum 과 carry\_ab 를 생성하고, 두 번째는 partial\_sum 과 cin 에서 최종 합 s 와 carry\_cin 을

생성한다. 두 단계의 캐리를

cout=carry\_ab|carry\_cin 으로 OR 하여 최종 캐리를 구한다.

**시험 조건.** (a,b,cin)의 8 개 조합을 순서대로 인가하였다. 20~40 ns 에서는 a=1'b0,b=1'b1 에서 cin=1'b0→1'b1 을 비교하였다.

**예상 결과.** s=a^b^cin,

cout=(a&b)|(a&cin)|(b&cin)이다. 0+1+0=1 에서 {cout,s}=2'b01, 0+1+1=2 에서 2'b10 을 예상하였다.

**관찰 결과.** cin=1'b0 일 때 s=1'b1,cout=1'b0, cin=1'b1 일 때 s=1'b0,cout=1'b1 이었다. 파형의 8 개 조합에서 {cout,s}는 2'd0,2'd1,2'd1,2'd2,2'd1,2'd2,2'd2,2'd3 으로 나타났다.

**해석.** 그림 2 에서 입력 캐리가 합에 반영되었고, 합이 2 이상일 때 캐리 출력이 발생하였다. 합과 캐리의 전환이 계산 결과와 일치하였다.

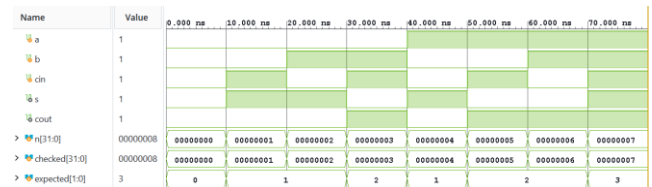


그림 2. 02. 전가산기의 입력·출력 및 예상값 비교 파형.

### 03. 4 비트 가산기

**설계 및 핵심 구조.** 두 4 비트 입력을 각각 5 비트로 확장한 뒤 더하도록 설계하였다. 핵심 구문은 assign {cout,s}={1'b0,a}+{1'b0,b}이다. 합의 하위 4 비트는 s 로, 최상위 비트는 cout 으로 출력하여 4 비트 범위를 넘는 캐리를 보존한다.

**시험 조건.** 그림 3 의 a=4'h5 구간에서 b 가 순차 증가하며, b=4'hA→4'hB 에서 합과 캐리를 비교하였다.

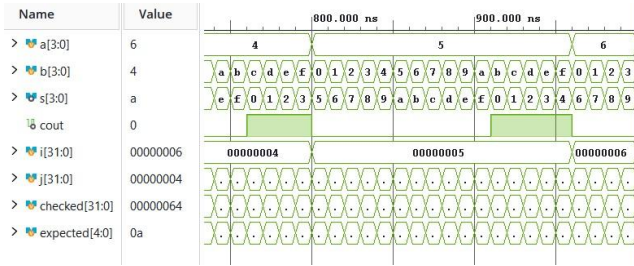
**예상 결과.** {cout,s}={1'b0,a}+{1'b0,b}이므로

4'h5+4'hA=5'h0F 에서 s=4'hF,cout=1'b0,

4'h5+4'hB=5'h10 에서 s=4'h0,cout=1'b1 을 예상하였다.

**관찰 결과.** a=4'h5,b=4'hA 에서 s=4'hF 였고, b=4'hB 에서 s=4'h0 으로 바뀌면서 cout 이 1'b1 로 상승하였다.

**해석.** 그림 3 에서 합의 4 비트 범위를 넘으면 하위 4 비트가 0 으로 바뀌고 상위 비트가 캐리로 출력되었다. 이 경계에서 합과 캐리가 예상값과 일치하였다.



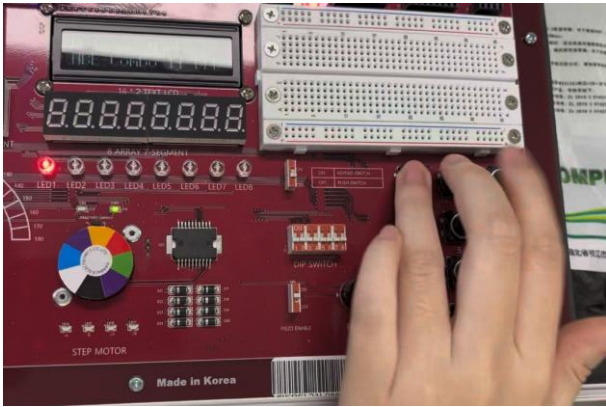


그림 6-1. 실험 06 MUX:  $s=2'b00$ , button1 에 연결된  $i[3]$  선택 조건의 입력 조작 및 LED1 점등 장면.

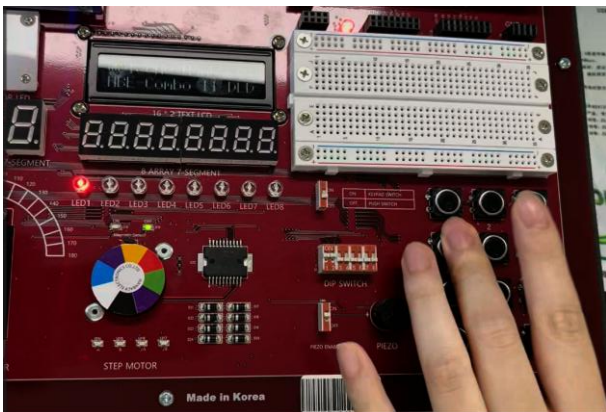


그림 6-2. 실험 06 MUX:  $s=2'b11$ , button4 에 연결된  $i[0]$  선택 조건의 입력 조작 및 LED1 점등 장면.

## 07. 1:8 DEMUX

**설계 및 핵심 구조.** 하나의 데이터 입력을 선택선  $s[2:0]$ 에 따라 8 개 출력 중 하나로 전달하도록 설계하였다.  $i=1'b1$  이면  $8'b10000000 \gg s$  로 활성 출력 위치를 이동시키고,  $i=1'b0$  이면  $o=8'h00$  으로 모든 출력을 비활성화한다. 선택값이 증가할수록 활성 비트가 상위 비트에서 하위 비트로 이동한다.

**시험 조건.**  $i=1'b0$  과  $i=1'b1$  각각에 대해  $s$  의 8 개 선택값을 순서대로 인가하였다. 대표적으로 80~100 ns 의  $i=1'b1, s=3'b000 \rightarrow 3'b001$  을 비교하였다.

**예상 결과.**  $i=1'b0$  이면  $o=8'h00$  이다.  $i=1'b1$  이면  $o=8'h80 \gg s$  이므로  $s=3'b000 \rightarrow 3'b001$  에서  $o=8'h80 \rightarrow 8'h40$  을 예상하였다.

**관찰 결과.** 0~80 ns 의  $i=1'b0$  에서는 모든 선택 조건의 출력이 8'h00 이었다. 80~160 ns 의  $i=1'b1$  에서는  $s$  의 순차 증가에 따라 출력이 8'h80, 8'h40, 8'h20, 8'h10, 8'h08, 8'h04, 8'h02, 8'h01 로 나타났다.

**해석.** 그림 7 에서 입력 데이터가 0 이면 모든 출력이 비활성이고, 1 이면 선택된 한 비트만 활성화되었다. 선택 신호에 따른 출력 이동으로 데이터 분배 기능을 확인하였다.

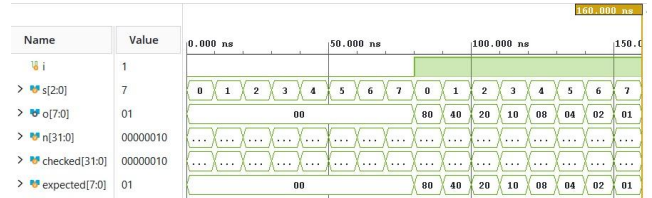


그림 7. 07. 1:8 DEMUX 의 입력·출력 및 예상값 비교 파형.

## 08. 8:3 인코더

**설계 및 핵심 구조.** 조합논리 case 문으로 단일 활성 입력을 3 비트 코드에 대응시켰다. 입력

8'h80, 8'h40, 8'h20, 8'h10, 8'h08, 8'h04, 8'h02, 8'h01 은 각각 코드

3'd0, 3'd1, 3'd2, 3'd3, 3'd4, 3'd5, 3'd6, 3'd7 에 대응한다. 일치하는 단일 활성 입력이 없으면 기본값 3'b000 을 출력하므로, 다중 활성 입력에서 우선순위를 선택하는 인코더가 아니다.

**시험 조건.** 10~40 ns 에서 입력

$i=8'h01 \rightarrow 8'h02 \rightarrow 8'h03$  을 비교하였다. 앞의 두 입력은 한 비트만 1 이고, 8'h03 은 두 비트가 1 인 조건이다.

**예상 결과.** 변환 규칙에 따라 8'h01  $\rightarrow a=3'd7$ , 8'h02  $\rightarrow a=3'd6$  을 예상하였다. 여러 비트가 1 인 8'h03 은 기본 출력 3'd0 으로 처리된다.

**관찰 결과.** 입력 8'h01, 8'h02, 8'h03 에서 출력 a 가 각각 3'd7, 3'd6, 3'd0 이었다. 파형의 입력 8'h00, 8'h04, 8'h08 에서도 출력이 각각 3'd0, 3'd5, 3'd4 로 나타났다.

**해석.** 그림 8 에서 단일 활성 입력은 해당 위치의 코드로 변환되고, 다중 활성 입력은 기본값으로 처리되었다. 따라서 이 회로는 우선순위 인코더가 아니며, 출력 0 만으로 유효 입력 8'h80 과 무효 입력을 구별할 수 없다.

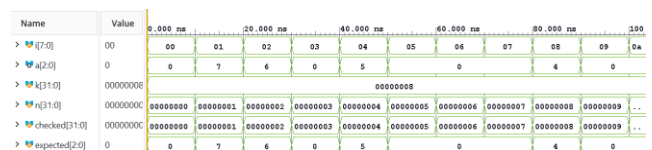


그림 8. 08. 8:3 인코더의 입력·출력 및 예상값 비교 파형.

## 09. 3:8 디코더

**설계 및 핵심 구조.** 세 입력을 {a,b,c} 순서의 3 비트 코드로 묶어 단일 활성 출력의 위치를 정하도록 설계하였다. RTL 의 핵심 관계는

$o=8'b00000001 \ll \{a,b,c\}$  이다. 입력 3'b000 은  $o[0]$ , 3'b111 은  $o[7]$  을 활성화한다.

**시험 조건.** {a,b,c} 를 3'b000 부터 3'b111 까지 순서대로 인가하였다. 30~50 ns 에서는 3'b011  $\rightarrow$  3'b100 경계를 비교하였다.

**예상 결과.**  $o=8'b00000001 \ll \{a,b,c\}$  이므로 대표 구간은 8'h08  $\rightarrow$  8'h10 을 예상하였다. 전체 입력 순서에서는 8'h01, 8'h02, 8'h04, 8'h08, 8'h10, 8'h20, 8'h40, 8'h80 이 예상된다.

**관찰 결과.** 입력 3'b011 에서 출력은 8'h08, 입력 3'b100 에서 8'h10 이었다. 파형의 8 개 입력 모두에서 출력이 예상한 순서로 이동하였다.

**해석.** 그림 9 에서 3 비트 입력 코드마다 대응하는 출력한 비트가 활성화되었다. 모든 표시 입력에서 디코딩 위치가 예상과 일치하였다.

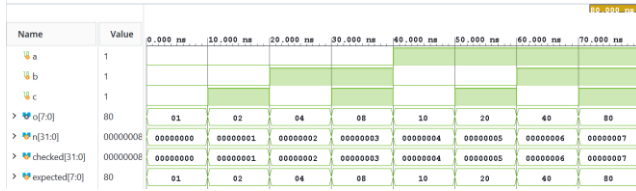


그림 9. 09. 3:8 디코더의 입력·출력 및 예상값 비교 파형.

## 10. 7-segment 디코더

**설계 및 핵심 구조.** 조합논리 case 문으로 4 비트 입력에 대응하는 세그먼트 패턴을 선택하였다. 출력 순서는 `seg_data[7:0]={a,b,c,d,e,f,g,dp}`이고, 1 이 세그먼트를 활성화하는 active-high 방식이다. 입력 4'h8 은 8'hFE, 입력 4'hF 는 8'h8E 에 대응하며 소수점은 비활성 상태이다.

**시험 조건.** 입력 bcd 를 4'h0 부터 4'hF 까지 순서대로 인가하였다. 보드 시험과 대응되는 입력 4'h8 의 80~90 ns 구간과 입력 4'hF 의 150~160 ns 구간을 각각 확인하였다.

**예상 결과.** 세그먼트 순서는

`seg_data[7:0]={a,b,c,d,e,f,g,dp}`이고 1 이 활성화 패턴이다. 입력 4'h8 은 8'hFE=8'b11111110 으로 a~g 를 켜고 소수점은 끈다. 입력 4'hF 는 8'h8E=8'b10001110 으로 a,e,f,g 를 켜고 b,c,d 및 소수점은 끈다.

**관찰 결과.** 입력 4'h8 에서 `seg_data=8'hFE`, 입력 4'hF 에서 `seg_data=8'h8E` 였다. 전체 입력의 출력은 8'hFC, 8'h60, 8'hDA, 8'hF2, 8'h66, 8'hB6, 8'hBE, 8'hE0, 8'hFE, 8'hF6, 8'hEE, 8'h3E, 8'h9A, 8'h7A, 8'h9E, 8'h8E 로, 파형의 expected 값과 일치하였다.

**해석.** 그림 10 에서 4 비트 입력에 따라 표시 문자에 필요한 세그먼트 패턴이 출력되었다. 입력 이름은 bcd 이지만 4'hA~4'hF 도 포함하여 16 진수 문자 변환을 확인하였다. 입력 8 과 F 의 각 구간에서도 예상한 패턴이 나타났다.

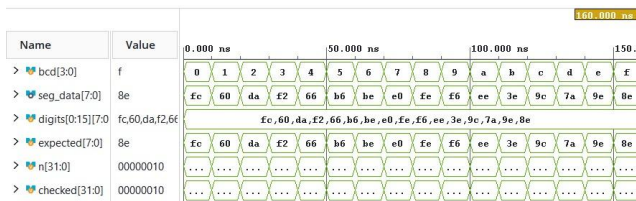


그림 10. 10. 7-segment 디코더의 입력·출력 및 예상값 비교 파형.

**실제 FPGA 보드 검증.** 입력 4'b1000 에서 숫자 8, 입력 4'b1111 에서 문자 F 가 표시될 것으로 예상하였다. 그림

10-1 과 10-2 의 실제 표시가 각각 8 과 F 로 나타나 예상 결과 및 시뮬레이션과 일치하였다.

표 2. 실험 10 의 예상 표시와 실제 표시 비교.

| 그림   | 입력 조건       | 시뮬레이션 패턴 | 예상 표시 | 실제 표시 | 비교 |
|------|-------------|----------|-------|-------|----|
| 10-1 | 4'b1000 (8) | 8'hFE    | 8     | 8     | 일치 |
| 10-2 | 4'b1111 (F) | 8'h8E    | F     | F     | 일치 |

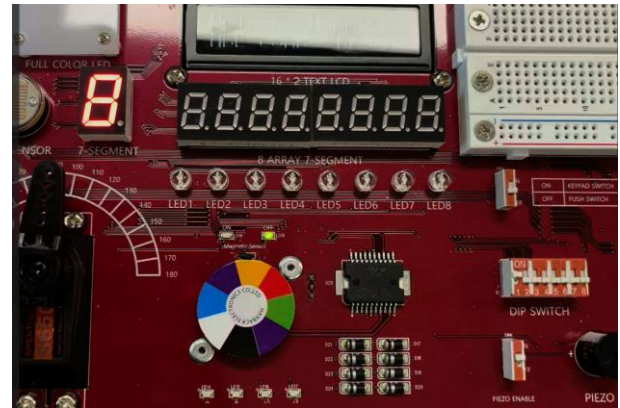


그림 10-1. 실험 10 7-segment 디코더: 입력 4'b1000 에서 실제 표시부에 8 이 나타난 장면.

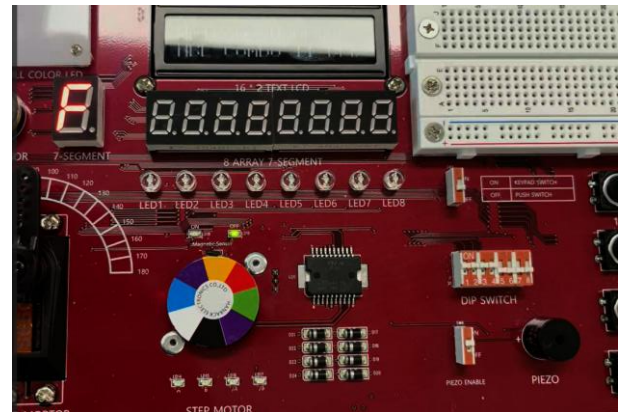


그림 10-2. 실험 10 7-segment 디코더: 입력 4'b1111 에서 실제 표시부에 F 가 나타난 장면.

## 3. Vivado 구현 및 검증 기록

### 3.1 구현 환경과 기록

기록을 확인할 수 있는 실험 01, 02, 04, 06, 08, 09, 10 의 7 개 프로젝트는 Vivado v2026.1 과 FPGA part xc7s75fpga484-1 을 사용하였다. 현재 소스의 설계 top 과 XDC 파일은 10 개 실험 모두에서 확인하였다. 이 7 개 프로젝트에는 synthesis 와 implementation 성공 로그, bitstream 파일, DRC report 및 timing report 가 남아 있다.

실험 03, 05, 07 의 구현 단계는 팀 분담에 따라 별도 환경에서 수행되었다. 원본 구현 로그와 산출물 일부가 해당 환경에 분산되어 있어, 아래의 DRC-timing 판독 결과는 기록이 확보된 7 개 프로젝트에 대해 정리하였다.

### 3.2 DRC 와 timing

확인 가능한 7 개 프로젝트의 routed DRC report 에서는 각각 error 0 건과 CFGBVS/CONFIG\_VOLTAGE 설정 관련 CFGBVS-1 warning 1 건이 확인되었다. 이 경고는 기능 시뮬레이션과 별개의 구현 기록이다. 또한 별도의 clock 및 timing constraint 가 없어 setup/hold 성능 판정에 사용할 timing 수치는 산출되지 않았다.

### 4. VS Code 와 Vivado 시뮬레이션 비교

VS Code 에서 수행한 Icarus 시뮬레이션과 Vivado 시뮬레이션을 비교하였다. 비교 가능한 실험 01, 02, 04, 06, 08, 09, 10 에서는 입력 순서, 출력, 검사 case 수 및 로그의 종료 시간이 일치하였다. 각 case 의 입력 전환 후 5 ns 인 안정 시점에서도 입력·출력 불일치가 없어 기능적으로 일치하였다.

실험 03, 05, 07 은 현재 확보된 Vivado 파형의 안정 구간을 예상값과 비교하였다.

### 5. 결론

LAB1 에서는 AND·OR·XOR 게이트, 가산기, 감산기, 비교기, MUX/DEMUX, 인코더/디코더, 7-segment decoder 등의 조합논리회로를 설계하고, 10 개 조합논리회로의 입력 조건과 예상 출력을 Vivado 시뮬레이션 파형으로 비교하였다. 분석한 구간에서 논리 연산, 캐리·빌림 발생, 대소 비교, 데이터 선택·분배 및 코드 변환이 각 회로의 규칙과 일치하였다.

실제 FPGA 에서는 조교가 선정한 실험 06 MUX 의 두 선택 조건에 대한 LED 점등과 실험 10 7-segment 디코더의 8-F 표시를 관찰하였다. 시뮬레이션은 신호의 논리 관계를 확인하고, 보드 사진은 시험한 조건에서의 물리적 동작을 확인하는 역할을 하였다.

이번 LAB 을 통해 가산기의 캐리 경계, 감산기의 빌림, 인코더의 다중 활성 입력 처리에서 출력의 의미를 구분할 수 있었다. 회로의 핵심 관계를 먼저 정리하고, 경계 조건과 대표 입력의 파형을 해석한 뒤 실제 보드 동작과 비교하는 설계·검증 과정을 익혔다.

### 부록 A. 전체 입력 검사 보조 자료

표 A-1 은 PASS 캡처의 검사 수와 종료 시간이다. 그림 A-1~A-7 은 전체 입력 조합 검사 완료를 보조하는 자료이며, 각 회로의 기능은 본문의 파형으로 해석하였다. 실험 03, 05, 07 은 별도 PASS 캡처가 없어 표에서 제외하였다.

| 실험               | 캡처의 검사 수 | 종료 시간   |
|------------------|----------|---------|
| 01 논리 게이트        | 4        | 40 ns   |
| 02 전가산기          | 8        | 80 ns   |
| 04 4 비트 감산기      | 256      | 2560 ns |
| 06 MUX           | 64       | 640 ns  |
| 08 인코더           | 256      | 2560 ns |
| 09 디코더           | 8        | 80 ns   |
| 10 7-segment 디코더 | 16       | 160 ns  |

```
LAB1_PASS logic_gate cases=4
$finish called at time : 40 ns : File "C:/Users/yunji/eece2/eece2_2026/LAB1/01_logic_gates/sim/tb_logic_gate_modern.vv" Line 19
```

그림 A-1. 실험 01 의 검사 수 및 종료 시간 기록.

```
LAB1_PASS full_adder cases=8
$finish called at time : 80 ns : File "C:/Users/yunji/eece2/eece2_2026/LAB1/02_full_adder/sim/tb_full_adder.vv" Line 33
```

그림 A-2. 실험 02 의 검사 수 및 종료 시간 기록.

```
LAB1_PASS sub_4bit cases=256
$finish called at time : 2560 ns : File "C:/Users/yunji/eece2/eece2_2026/LAB1/04_4bit_subtractor/sim/tb_sub_4bit.vv" Line 39
```

그림 A-3. 실험 04 의 검사 수 및 종료 시간 기록.

```
LAB1_PASS mux_4x1 cases=64
$finish called at time : 640 ns : File "C:/Users/yunji/eece2/eece2_2026/LAB1/06_mux/sim/tb_mux_4x1.vv" Line 37
```

그림 A-4. 실험 06 의 검사 수 및 종료 시간 기록.

```
LAB1_PASS encoder8x3 cases=256
$finish called at time : 2560 ns : File "C:/Users/yunji/eece2/eece2_2026/LAB1/08_encoder/sim/tb_encoder8x3.vv" Line 40
```

그림 A-5. 실험 08 의 검사 수 및 종료 시간 기록.

```
LAB1_PASS decoder3x8 cases=8
$finish called at time : 80 ns : File "C:/Users/yunji/eece2/eece2_2026/LAB1/09_decoder/sim/tb_decoder3x8.vv" Line 35
```

그림 A-6. 실험 09 의 검사 수 및 종료 시간 기록.

```
LAB1_PASS seg_decoder cases=16
$finish called at time : 160 ns : File "C:/Users/yunji/eece2/eece2_2026/LAB1/10_7seg_decoder/sim/tb_seg_decoder.vv" Line 53
```

그림 A-7. 실험 10 의 검사 수 및 종료 시간 기록.

### 부록 B. FPGA programming 보조 자료

그림 B-1 의 Hardware Device Properties 에는 장치 xc7s75\_0, Part xc7s75, 상태 Programmed 가 표시된다. 이 화면은 FPGA programming 수행의 보조 자료이다. 회로 기능은 본문의 시뮬레이션 파형과 보드 동작 사진으로 확인하였다.

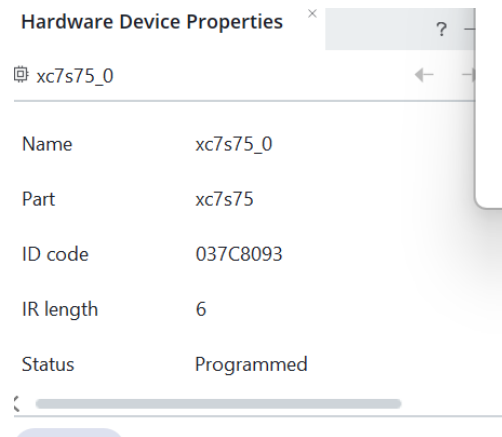


그림 B-1. FPGA 장치의 Programmed 상태 확인 화면.

## 부록 c. 제출 및 구현 추적 기록

최종 code commit:

246d6b621ce5b7a30d8a71a98c3e03693e69c9de

표 C-1. 실험별 bitstream 기록

| 실험               | bitstream 기록                                                      |
|------------------|-------------------------------------------------------------------|
| 01 논리 게이트        | LAB1/01_logic_gates/logic_gate.runs/impl_1/logic_gate.bit         |
| 02 전가산기          | LAB1/02_full_adder/full_adder.runs/impl_1/full_adder.bit          |
| 03 4 비트 가산기      | 팀원 수행 환경에서 생성, 현재 작성 환경에는 원본 미보관                                  |
| 04 4 비트 감산기      | LAB1/04_4bit_subtractor/4bits_subtractor.runs/impl_1/sub_4bit.bit |
| 05 4 비트 비교기      | 팀원 수행 환경에서 생성, 현재 작성 환경에는 원본 미보관                                  |
| 06 MUX           | LAB1/06_mux/mux.runs/impl_1/mux_4x1.bit                           |
| 07 DEMUX         | 팀원 수행 환경에서 생성, 현재 작성 환경에는 원본 미보관                                  |
| 08 인코더           | LAB1/08_encoder/08_encoder.runs/impl_1/encoder8x3.bit             |
| 09 디코더           | LAB1/09_decoder/decoder.runs/impl_1/decoder3x8.bit                |
| 10 7-segment 디코더 | LAB1/10_7seg_decoder/10_7seg_decoder.runs/impl_1/seg_decoder.bit  |

## 참고문헌

[1] M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed., Pearson, 2013.